

Processamento Digital de Imagens e Visão Computacional

Abordagem Prática com Python

Francisco de Assis Zampirolli

2026-05-04

Índice

PDI e VC — Python	1
Organização	1
 I Parte I — Fundamentos de PDI	 2
1 Representação de Imagens Digitais	3
1.1 Objetivos	3
1.2 O que é uma Imagem Digital?	3
1.3 Configuração do Ambiente	4
1.4 1.3 Lendo e Exibindo Imagens	5
1.5 1.4 Tipos de Imagem	7
1.5.1 1.4.1 Conversão para Tons de Cinza	7
1.5.2 1.4.2 Limiarização (Binarização)	8
1.6 1.5 Acesso a Pixels	8
1.7 1.6 Resumo	9
1.7.1 Questões de Revisão	9
 2 Representação de Imagens Digitais cap2	 11
2.1 Objetivos	11
2.2 O que é uma Imagem Digital?	11
2.3 Configuração do Ambiente	12
2.4 1.3 Lendo e Exibindo Imagens	13
2.5 1.4 Tipos de Imagem	15
2.5.1 1.4.1 Conversão para Tons de Cinza	15
2.5.2 1.4.2 Limiarização (Binarização)	16
2.6 1.5 Acesso a Pixels	16

<i>ÍNDICE</i>	ii
---------------	----

2.7 1.6 Resumo	17
2.7.1 Questões de Revisão	17

Appendices	19
-------------------	-----------

Referências	19
--------------------	-----------

PDI e VC — Python

Bem-vindo ao livro de PDI e Visão Computacional — versão Python / Português.

Organização

- **Parte I — Fundamentos de PDI** — Representação, histogramas, filtragem, morfologia
 - **Parte II — Visão Computacional** — Segmentação, descritores, detecção, deep learning
-

Parte I

Parte I — Fundamentos de PDI

Capítulo 1

Representação de Imagens Digitais



Este capítulo apresenta os fundamentos de **Processamento Digital de Imagens (PDI)**, seguindo a metodologia de (Zampirolli; Josko; Teubl; Kurashima; Lopes, 2025) (que é uma versão estendida de (Zampirolli; Josko; Teubl; Kurashima, 2024), que recebeu prêmio de melhor artigo na “Trilha de Recursos e Ambientes Educacionais” na EduComp24) com a biblioteca `morph.py`.

1.1 Objetivos

Ao final deste capítulo, o aluno será capaz de:

- Compreender a representação matemática de imagens $f(x, y)$
- Ler, exibir e salvar imagens
- Trabalhar com imagens binárias, tons de cinza e coloridas (RGB)
- Acessar e modificar pixels individualmente

1.2 O que é uma Imagem Digital?

Uma imagem digital é uma função discreta:

$$f : \mathbb{Z}^2 \rightarrow \mathbb{Z}^k \quad (1.1)$$

onde (x, y) são coordenadas espaciais e $f(x, y)$ é a intensidade do pixel. Para imagens em **tons de cinza** $k = 1$ com valores em $[0, 255]$; para imagens **coloridas** (RGB) $k = 3$, ver Figura 2.1.



Figura 1.1: Exemplo de Transformação Não Linear do Espaço de Entrada para o Espaço de Características.

1.3 Configuração do Ambiente

```
# Instalação (Google Colab - executar uma vez)
# !pip install opencv-python-headless scikit-image matplotlib numpy

import sys
import importlib
```

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

sys.path.insert(0, '.././morph')
import morph
importlib.reload(morph)
from morph import mm

print(f'OpenCV {cv2.__version__} | NumPy {np.__version__}')
print("Carregando imagem...")
print(help(mm.show))
```

OpenCV 4.12.0 | NumPy 2.2.6

Carregando imagem...

Help on function show in module morph:

```
show(*args, title=None)
    This function will draw images f
    input: <*args> set of images f_i, where i>0 is binary image
           <title> optional string title for the plot
    output: image drawing
    Example:
    f1, f2 = np.zeros((100, 100,3)), np.zeros((100, 100))
    f2[50:60, 50:60] = 1
    mm.show(f1, f2, title='Exemplo')
```

None

1.4 1.3 Lendo e Exibindo Imagens

A operação mais básica em PDI é a leitura de uma imagem. A função `mm.read()` da `morph.py` aceita caminhos locais e URLs.

```
import urllib.request, os

# Baixar imagem de exemplo
```



```
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/lena.jpg'
os.makedirs('imagens', exist_ok=True)
if not os.path.exists('imagens/lena.jpg'):
    urllib.request.urlretrieve(url, 'imagens/lena.jpg')

# Ler imagem com morph.py
img = mm.read('imagens/lena.jpg')

print(f'Shape : {img.shape}')    # (altura, largura, canais)
print(f'Dtype  : {img.dtype}')    # uint8
print(f'Min/Max: {img.min()} / {img.max()}')

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
mm.show(img_rgb, title='Imagem original')
```

Shape : (512, 512, 3)

Dtype : uint8

Min/Max: 0 / 255

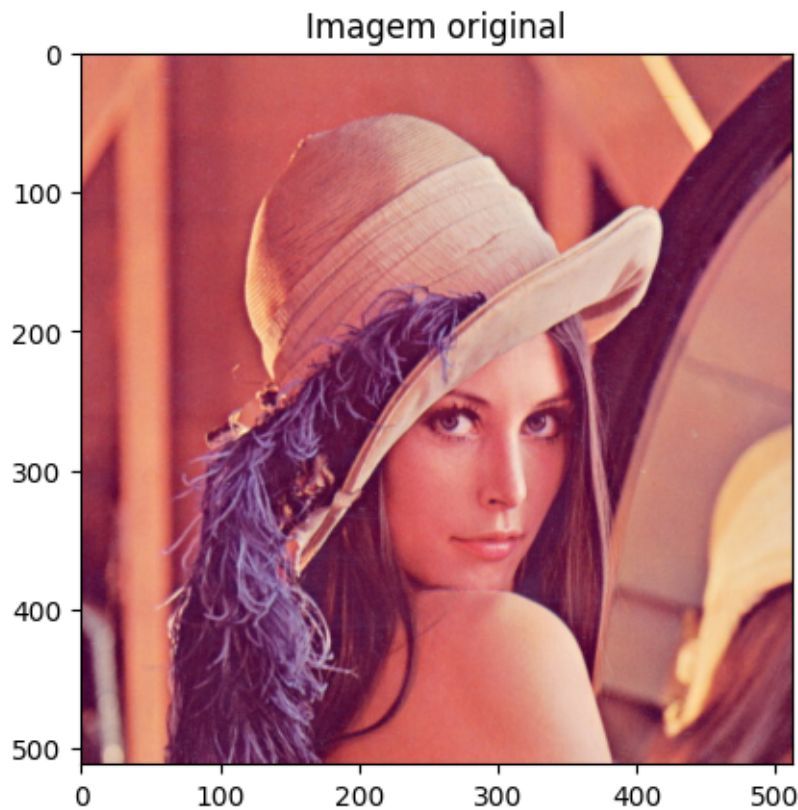


Figura 1.2: MVS Linear — Margem Máxima entre Íris Setosa e Versicolor (atributos de pétala).

1.5 1.4 Tipos de Imagem

1.5.1 1.4.1 Conversão para Tons de Cinza

A conversão de RGB para tons de cinza aplica pesos perceptuais definidos pela ITU-R BT.601:

$$g = 0.299 R + 0.587 G + 0.114 B \quad (1.2)$$

```
# Converter para tons de cinza  
img_gray = mm.gray(img)
```

```
print(f'Shape cinza: {img_gray.shape}')

mm.show([img, img_gray], title=['Original (RGB)', 'Tons de Cinza'])
```

1.5.2 1.4.2 Limiarização (Binarização)

A limiarização converte uma imagem em tons de cinza para binária:

$$g(x, y) = \begin{cases} 255 & \text{se } f(x, y) \geq T \\ 0 & \text{caso contrário} \end{cases} \quad (1.3)$$

O método de **Otsu** (Otsu, 1979) determina T automaticamente maximizando a variância inter-classes do histograma.

```
# Limiarização com T fixo
T = 128
img_bin = mm.threshold(img_gray, T)

# Limiarização Otsu (T automático)
_, img_otsu = cv2.threshold(img_gray, 0, 255,
                             cv2.THRESH_BINARY + cv2.THRESH_OTSU)

print(f'Valores únicos (T={T}): {sorted(set(img_bin.flatten()))}')

mm.show([img_gray, img_bin, img_otsu],
         title=[f'Cinza', f'Binária (T={T})', 'Otsu'])
```

1.6 1.5 Acesso a Pixels

Em Python/NumPy, o acesso a pixels usa indexação de arrays `img[linha, coluna]`.

```
r, c = 100, 100

# Acessar pixel
pixel_cinza = img_gray[r, c]
pixel_rgb = img[r, c] # array [R, G, B]
```

```
print(f'Pixel cinza ({r},{c}): {pixel_cinza}')
```

```
print(f'Pixel RGB   ({r},{c}): R={pixel_rgb[0]}  G={pixel_rgb[1]}  B={pixel_rgb[2]}')
```

```
# Imagem sintética 5x5 com pixel central branco
```

```
syn = np.zeros((5, 5), dtype=np.uint8)
```

```
syn[2, 2] = 255
```

```
print('\nMatriz 5x5:')
```

```
print(syn)
```

1.7 1.6 Resumo

Neste capítulo foram apresentados os fundamentos de representação de imagens digitais:

- Imagem digital = função $f(x, y)$ mapeando coordenadas para intensidades
- Tipos: binária ($k = 1, \{0, 255\}$), tons de cinza ($k = 1, [0, 255]$), RGB ($k = 3$)
- `morph.py`: biblioteca didática que simplifica operações de PDI em Python
- Limiarização: operação fundamental de binarização

O Capítulo 2 estuda **histogramas e equalização**.

1.7.1 Questões de Revisão

1. Qual a diferença entre imagem em tons de cinza e imagem binária?
2. Como o método de Otsu determina o limiar T automaticamente?
3. O que representa `img.shape` para uma imagem NumPy RGB?

Referências

OTSU, Nobuyuki. A threshold selection method from gray-level histograms. **IEEE Transactions on Systems, Man, and Cybernetics**, IEEE, v. 9, n. 1, p. 62–66, jan. 1979. ISSN 0018-9472. DOI: [10.1109/TSMC.1979.4310076](https://doi.org/10.1109/TSMC.1979.4310076). Disponível em: <https://ieeexplore.ieee.org/document/4310076>.

ZAMPIROLI, Francisco de Assis; JOSKO, João Marcelo Borovina; TEUBL, Fernando; KURASHIMA, Celso Setsuo. A Practical Digital Image Processing Course with morph.py. *In*: ANAIS do IV Simpósio Brasileiro de Educação em Computação (EduComp 2024). Evento Online: Sociedade Brasileira de Computação (SBC), 2024. p. 304–313. DOI: [10.5753/educomp.2024.237274](https://doi.org/10.5753/educomp.2024.237274). Disponível em: <https://sol.sbc.org.br/index.php/educomp/article/view/28199>.

ZAMPIROLI, Francisco de Assis; JOSKO, João Marcelo Borovina; TEUBL, Fernando; KURASHIMA, Celso Setsuo; LOPES, Renato de Avila. Teaching Hands-On Digital Image Processing with morph.py: Methods and Comprehensive Results. **Revista Brasileira de Informática na Educação**, v. 33, p. 990–1026, ago. 2025. DOI: [10.5753/rbie.2025.5009](https://doi.org/10.5753/rbie.2025.5009). Disponível em: <https://journals-sol.sbc.org.br/index.php/rbie/article/view/5009>.

Capítulo 2

Representação de Imagens Digitais cap2

 Executar Colab  Abrir si-md2 GitHub

Este capítulo apresenta os fundamentos de **Processamento Digital de Imagens (PDI)**, seguindo a metodologia de (Zampirolli; Josko; Teubl; Kurashima; Lopes, 2025) (que é uma versão estendida de (Zampirolli; Josko; Teubl; Kurashima, 2024), que recebeu prêmio de melhor artigo na “Trilha de Recursos e Ambientes Educacionais” na EduComp24) com a biblioteca `morph.py`.

2.1 Objetivos

Ao final deste capítulo, o aluno será capaz de:

- Compreender a representação matemática de imagens $f(x, y)$
- Ler, exibir e salvar imagens
- Trabalhar com imagens binárias, tons de cinza e coloridas (RGB)
- Acessar e modificar pixels individualmente

2.2 O que é uma Imagem Digital?

Uma imagem digital é uma função discreta:

$$f : \mathbb{Z}^2 \rightarrow \mathbb{Z}^k \quad (2.1)$$

onde (x, y) são coordenadas espaciais e $f(x, y)$ é a intensidade do pixel. Para imagens em **tons de cinza** $k = 1$ com valores em $[0, 255]$; para imagens **coloridas** (RGB) $k = 3$, ver Figura 2.1.



Figura 2.1: Exemplo de Transformação Não Linear do Espaço de Entrada para o Espaço de Características.

2.3 Configuração do Ambiente

```
# Instalação (Google Colab - executar uma vez)
# !pip install opencv-python-headless scikit-image matplotlib numpy

import sys
import importlib
```

```

import numpy as np
import cv2
import matplotlib.pyplot as plt

sys.path.insert(0, '.././morph')
import morph
importlib.reload(morph)
from morph import mm

print(f'OpenCV {cv2.__version__} | NumPy {np.__version__}')
print("Carregando imagem...")
print(help(mm.show))

```

OpenCV 4.12.0 | NumPy 2.2.6

Carregando imagem...

Help on function show in module morph:

```

show(*args, title=None)
    This function will draw images f
    input: <*args> set of images f_i, where i>0 is binary image
           <title> optional string title for the plot
    output: image drawing
    Example:
    f1, f2 = np.zeros((100, 100,3)), np.zeros((100, 100))
    f2[50:60, 50:60] = 1
    mm.show(f1, f2, title='Exemplo')

```

None

2.4 1.3 Lendo e Exibindo Imagens

A operação mais básica em PDI é a leitura de uma imagem. A função `mm.read()` da `morph.py` aceita caminhos locais e URLs.

```

import urllib.request, os

# Baixar imagem de exemplo

```



```
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/lena.jpg'
os.makedirs('imagens', exist_ok=True)
if not os.path.exists('imagens/lena.jpg'):
    urllib.request.urlretrieve(url, 'imagens/lena.jpg')

# Ler imagem com morph.py
img = mm.read('imagens/lena.jpg')

print(f'Shape : {img.shape}')      # (altura, largura, canais)
print(f'Dtype : {img.dtype}')      # uint8
print(f'Min/Max: {img.min()} / {img.max()}')

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
mm.show(img_rgb, title='Imagem original')
```

```
Shape : (512, 512, 3)
Dtype : uint8
Min/Max: 0 / 255
```

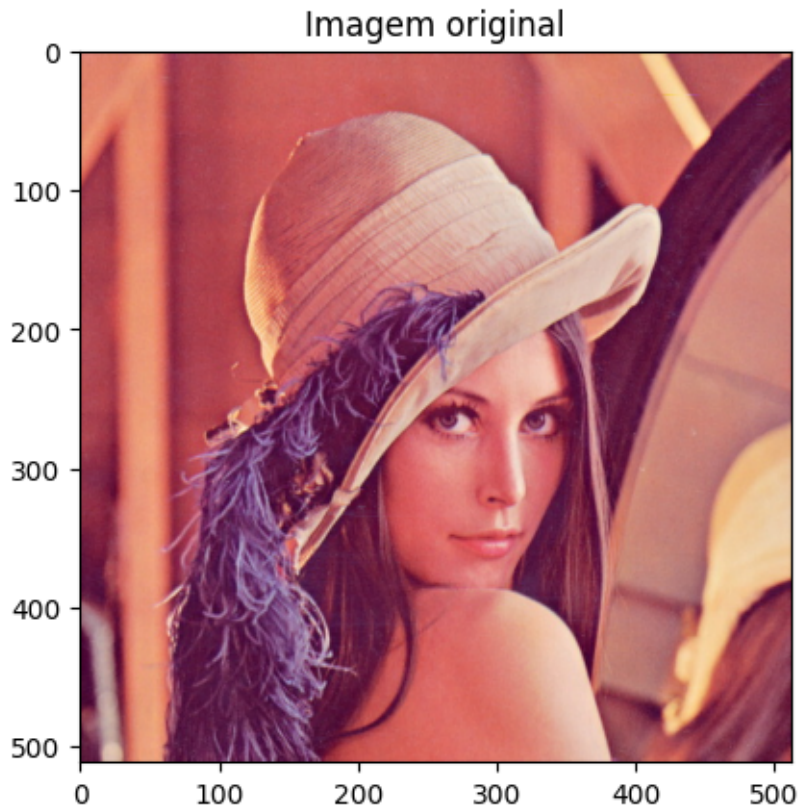


Figura 2.2: MVS Linear — Margem Máxima entre Íris Setosa e Versicolor (atributos de pétala).

2.5 1.4 Tipos de Imagem

2.5.1 1.4.1 Conversão para Tons de Cinza

A conversão de RGB para tons de cinza aplica pesos perceptuais definidos pela ITU-R BT.601:

$$g = 0.299 R + 0.587 G + 0.114 B \quad (2.2)$$

```
# Converter para tons de cinza
img_gray = mm.gray(img)
```

```
print(f'Shape cinza: {img_gray.shape}')

mm.show([img, img_gray], title=['Original (RGB)', 'Tons de Cinza'])
```

2.5.2 1.4.2 Limiarização (Binarização)

A limiarização converte uma imagem em tons de cinza para binária:

$$g(x, y) = \begin{cases} 255 & \text{se } f(x, y) \geq T \\ 0 & \text{caso contrário} \end{cases} \quad (2.3)$$

O método de **Otsu** (Otsu, 1979) determina T automaticamente maximizando a variância inter-classes do histograma.

```
# Limiarização com T fixo
T = 128
img_bin = mm.threshold(img_gray, T)

# Limiarização Otsu (T automático)
_, img_otsu = cv2.threshold(img_gray, 0, 255,
                             cv2.THRESH_BINARY + cv2.THRESH_OTSU)

print(f'Valores únicos (T={T}): {sorted(set(img_bin.flatten()))}')

mm.show([img_gray, img_bin, img_otsu],
         title=[f'Cinza', f'Binária (T={T})', 'Otsu'])
```

2.6 1.5 Acesso a Pixels

Em Python/NumPy, o acesso a pixels usa indexação de arrays `img[linha, coluna]`.

```
r, c = 100, 100

# Acessar pixel
pixel_cinza = img_gray[r, c]
pixel_rgb = img[r, c] # array [R, G, B]
```

```

print(f'Pixel cinza ({r},{c}): {pixel_cinza}')
print(f'Pixel RGB   ({r},{c}): R={pixel_rgb[0]}  G={pixel_rgb[1]}  B={pixel_rgb[2]}

# Imagem sintética 5x5 com pixel central branco
syn = np.zeros((5, 5), dtype=np.uint8)
syn[2, 2] = 255
print('\nMatriz 5x5:')
print(syn)

```

2.7 1.6 Resumo

Neste capítulo foram apresentados os fundamentos de representação de imagens digitais:

- Imagem digital = função $f(x, y)$ mapeando coordenadas para intensidades
- Tipos: binária ($k = 1, \{0, 255\}$), tons de cinza ($k = 1, [0, 255]$), RGB ($k = 3$)
- `morph.py`: biblioteca didática que simplifica operações de PDI em Python
- Limiarização: operação fundamental de binarização

O Capítulo 2 estuda **histogramas e equalização**.

2.7.1 Questões de Revisão

1. Qual a diferença entre imagem em tons de cinza e imagem binária?
2. Como o método de Otsu determina o limiar T automaticamente?
3. O que representa `img.shape` para uma imagem NumPy RGB?

Referências

OTSU, Nobuyuki. A threshold selection method from gray-level histograms. **IEEE Transactions on Systems, Man, and Cybernetics**, IEEE, v. 9, n. 1, p. 62–66, jan. 1979. ISSN 0018-9472. DOI: [10.1109/TSMC.1979.4310076](https://doi.org/10.1109/TSMC.1979.4310076). Disponível em: <https://ieeexplore.ieee.org/document/4310076>.

ZAMPIROLI, Francisco de Assis; JOSKO, João Marcelo Borovina; TEUBL, Fernando; KURASHIMA, Celso Setsuo. A Practical Digital Image Processing Course with morph.py. *In*: ANAIS do IV Simpósio Brasileiro de Educação em Computação (EduComp 2024). Evento Online: Sociedade Brasileira de Computação (SBC), 2024. p. 304–313. DOI: [10.5753/educomp.2024.237274](https://doi.org/10.5753/educomp.2024.237274). Disponível em: <https://sol.sbc.org.br/index.php/educomp/article/view/28199>.

ZAMPIROLI, Francisco de Assis; JOSKO, João Marcelo Borovina; TEUBL, Fernando; KURASHIMA, Celso Setsuo; LOPES, Renato de Avila. Teaching Hands-On Digital Image Processing with morph.py: Methods and Comprehensive Results. **Revista Brasileira de Informática na Educação**, v. 33, p. 990–1026, ago. 2025. DOI: [10.5753/rbie.2025.5009](https://doi.org/10.5753/rbie.2025.5009). Disponível em: <https://journals-sol.sbc.org.br/index.php/rbie/article/view/5009>.

Referências

OTSU, Nobuyuki. A threshold selection method from gray-level histograms. **IEEE Transactions on Systems, Man, and Cybernetics**, IEEE, v. 9, n. 1, p. 62–66, jan. 1979. ISSN 0018-9472. DOI: [10.1109/TSMC.1979.4310076](https://doi.org/10.1109/TSMC.1979.4310076). Disponível em: <https://ieeexplore.ieee.org/document/4310076>.

ZAMPIROLI, Francisco de Assis; JOSKO, João Marcelo Borovina; TEUBL, Fernando; KURASHIMA, Celso Setsuo. A Practical Digital Image Processing Course with morph.py. *In*: ANAIS do IV Simpósio Brasileiro de Educação em Computação (EduComp 2024). Evento Online: Sociedade Brasileira de Computação (SBC), 2024. p. 304–313. DOI: [10.5753/educomp.2024.237274](https://doi.org/10.5753/educomp.2024.237274). Disponível em: <https://sol.sbc.org.br/index.php/educomp/article/view/28199>.

ZAMPIROLI, Francisco de Assis; JOSKO, João Marcelo Borovina; TEUBL, Fernando; KURASHIMA, Celso Setsuo; LOPES, Renato de Avila. Teaching Hands-On Digital Image Processing with morph.py: Methods and Comprehensive Results. **Revista Brasileira de Informática na Educação**, v. 33, p. 990–1026, ago. 2025. DOI: [10.5753/rbie.2025.5009](https://doi.org/10.5753/rbie.2025.5009). Disponível em: <https://journals-sol.sbc.org.br/index.php/rbie/article/view/5009>.